A mobile application for augmenting technical documentation using AR and AI

CS310: Third Year Project

Ani Bitri

2287990

Supervisors: Dr. Paris Giampouras, John McNamara

Department of Computer Science

University of Warwick and IBM

January 2026

Contents

1 Introduction **4**
1.1 Preface . . . 4
1.2 Glossary . . . 4
1.3 Objective and Scope . . . 4

2 Design **4**
2.1 Architectural Pattern . . . 4
2.2 Tools . . . 4
2.2.1 Version Control . . . 4
2.2.2 Testing Frameworks . . . 4
2.2.3 AI Models and Libraries . . . 4
2.2.4 Devices . . . 4
2.3 UI Construction . . . 4
2.3.1 Design Principles and User Experience . . . 4
2.3.2 Mockups and Prototypes . . . 4
2.3.3 Mobile Application (React Native) . . . 4
2.3.4 Web Application (React) . . . 4
2.4 Data Flow . . . 4
2.4.1 Document Upload and Processing Pipeline . . . 4
2.4.2 AR Generation Pipeline . . . 4
2.4.3 AI Query Pipeline . . . 4

3 Implementation **4**
3.1 Backend Implementation . . . 5
3.1.1 Model Manager . . . 5
3.1.2 Preprocessing Service . . . 5
3.1.3 Granite Vision Service . . . 5
3.1.4 Component Detection Algorithm . . . 5
3.1.5 Granite AI Service . . . 5
3.1.6 Prompt Engineering . . . 5
3.1.7 API Routes . . . 5
3.2 Frontend Implementation . . . 5
3.2.1 Mobile Application (React Native) . . . 5
3.2.2 Web Application (React) . . . 5
3.2.3 Shared Code Module . . . 5

4 Testing and Results **5**
4.1 Testing Strategy . . . 6
4.1.1 Unit Testing with Pytest . . . 6
4.1.2 Test Fixtures and Mock Data . . . 6
4.2 Backend Tests . . . 6
4.2.1 Component Detection Tests . . . 6
4.2.2 Vision Service Tests . . . 6
4.2.3 AI Service Tests . . . 6
4.2.4 Preprocessing Tests . . . 6
4.2.5 Upload and Routing Tests . . . 6
4.2.6 Health and Security Tests . . . 6
4.3 Integration Tests . . . 6
4.4 Device Testing . . . 6
4.5 Results and Analysis . . . 6
4.5.1 Component Detection Accuracy . . . 7
4.5.2 Time Performance Metrics . . . 7
4.5.3 AI Response Quality Assessment . . . 7
4.5.4 User Feedback and Usability Assessment . . . 7

5 Evaluation **7**
5.1 Objective Achievement . . . 7
5.2 Requirement Satisfaction . . . 7
5.2.1 Functional Requirements Evaluation . . . 7
5.2.2 Non-Functional Requirements Evaluation . . . 7
5.3 Strengths and Limitations . . . 7

6 Further Improvements and Future Work **7**

7 Professional, Ethical, Legal and Social Issues **7**

1 Introduction

1.1 Preface

1.2 Glossary

Key terms and definitions relevant to the project

1.3 Objective and Scope

Clear statement of the project objectives, deliverables, and scope.

2 Design

Overview of the design decisions made during the project, including architectural choices, tools and technologies used, and the rationale behind them.

2.1 Architectural Pattern

Description of the overall architecture, including the separation of concerns between frontend and backend, and the use of microservices for modularity.

2.2 Tools

Discussion of the development tools, frameworks, and libraries used in the project, along with justifications for their selection.

2.2.1 Version Control

2.2.2 Testing Frameworks

2.2.3 AI Models and Libraries

2.2.4 Devices

2.3 UI Construction

Presentation of the user interface design, including wireframes, mockups, and the user experience flow. Discussion of design principles followed to ensure usability and accessibility.

2.3.1 Design Principles and User Experience

2.3.2 Mockups and Prototypes

2.3.3 Mobile Application (React Native)

2.3.4 Web Application (React)

2.4 Data Flow

Detailed explanation of the data flow through the system, from document upload to AR generation and AI querying. This includes the orchestration of services and the interaction between different components.

2.4.1 Document Upload and Processing Pipeline

PDF extraction, image filtering, text extraction, and preparation for AR generation and AI querying.

2.4.2 AR Generation Pipeline

AR model loading, scene initialization, and rendering strategies.

2.4.3 AI Query Pipeline

Context building, progressive truncation for OOM handling, and response cleaning strategies.

3 Implementation

A comprehensive overview of the implementation details, including code structure, key algorithms, and the integration of different components.

3.1 Backend Implementation

3.1.1 Model Manager

Detailed implementation of the Model Manager, including the singleton pattern for efficient resource management, hardware-aware loading strategies for optimal performance, and mechanisms for monitoring VRAM usage and recovering from OOM errors.

3.1.2 Preprocessing Service

Implementation of the preprocessing service, which handles PDF extraction using PyMuPDF, image filtering to remove irrelevant visuals, and text extraction using Docling. This service orchestrates the preprocessing pipeline to prepare data for AR generation and AI querying.

3.1.3 Granite Vision Service

Implementation of the Granite Vision Service, which performs image analysis tasks such as component extraction. This includes strategies for resizing images to fit token limits while preserving important details for accurate AR generation and AI querying.

3.1.4 Component Detection Algorithm

Detailed implementation of the component detection algorithm.

3.1.5 Granite AI Service

Description of the Granite AI Service, which handles document Q&A by building context from extracted text and images. This includes strategies for progressive OOM truncation to ensure responses can be generated even with large documents, and response cleaning to improve the quality of AI-generated answers.

3.1.6 Prompt Engineering

Explanation of the prompt engineering strategies used to optimize AI responses, including specific prompts for component labeling, connection identification, AR extraction, and iterative refinement of responses based on user feedback.

3.1.7 API Routes

Overview of the API routes implemented in the Flask application, including endpoints for document upload, AR generation, vision analysis, AI chat, document processing, and health checks. Each route is designed to handle specific tasks while ensuring efficient communication between the frontend and backend.

3.2 Frontend Implementation

Key implementation details of the frontend applications, including the mobile application built with React Native and the web application built with React. This section also covers the shared code module used for common functionality across both platforms.

3.2.1 Mobile Application (React Native)

Structure and key features of the mobile application, including the user interface components, state management, and integration with the backend API for document upload, AR visualization, and AI querying.

3.2.2 Web Application (React)

Structure and key features of the web application, including the user interface components, state management, and integration with the backend API for document upload, AR visualization, and AI querying.

3.2.3 Shared Code Module

Description of the shared code module that contains common functionality used by both the mobile and web applications, such as API communication, data formatting, and utility functions.

4 Testing and Results

Full overview of the testing strategy, including unit tests, integration tests, and device testing. This section also presents the results of the tests, including metrics on component detection accuracy, time performance, AI response quality, and user feedback.

4.1 Testing Strategy

Recap of the testing strategy, including the types of tests conducted and the rationale behind them.

4.1.1 Unit Testing with Pytest

Coverage of unit tests implemented using Pytest for the backend services, including tests for the Model Manager, Preprocessing Service, Granite Vision Service, Component Detection Algorithm, Granite AI Service, Prompt Engineering, and API Routes.

4.1.2 Test Fixtures and Mock Data

Description of the test fixtures and mock data used to simulate various scenarios during testing, such as different document types, image complexities, and AI response variations.

4.2 Backend Tests

Tests for individual backend components to ensure they function correctly in isolation, including tests for model loading, PDF processing, image analysis, AI response generation, and API endpoint functionality.

4.2.1 Component Detection Tests

Visual and quantitative tests to evaluate the accuracy of the component detection algorithm, including metrics such as precision, recall, and F1 score.

4.2.2 Vision Service Tests

Structural tests for the Granite Vision Service to ensure it correctly processes images and extracts relevant information for AR generation and AI querying.

4.2.3 AI Service Tests

Tests for the Granite AI Service to evaluate the quality of AI responses, including context building, OOM handling, and response cleaning.

4.2.4 Preprocessing Tests

Tests for the preprocessing pipeline to ensure accurate PDF extraction, image filtering, and text extraction.

4.2.5 Upload and Routing Tests

Tests for the document upload functionality and API routing to ensure proper communication between the frontend and backend.

4.2.6 Health and Security Tests

Tests to ensure the health of the backend services and the security of API endpoints, including authentication and input validation tests.

4.3 Integration Tests

Tests that evaluate the interaction between different backend components and the frontend applications, ensuring that the entire system works cohesively to achieve the desired functionality.

4.4 Device Testing

Tests conducted on actual mobile devices and web browsers to evaluate the performance and usability of the applications in real-world scenarios, including tests for AR visualization, AI querying, and overall user experience.

4.5 Results and Analysis

Evaluation of the test results, including metrics on component detection accuracy, time performance, AI response quality, and user feedback. This section also includes an analysis of the results to identify strengths, weaknesses, and areas for improvement.

4.5.1 Component Detection Accuracy

Metrics such as precision and accuracy to evaluate the performance of the component detection algorithm.

4.5.2 Time Performance Metrics

Analysis of the time taken for various processes, including PDF processing, AR generation, and AI response generation.

4.5.3 AI Response Quality Assessment

Evaluation of the quality of AI responses based on relevance, accuracy, and user satisfaction.

4.5.4 User Feedback and Usability Assessment

Summary of user feedback collected during testing, including insights on usability, functionality, and overall user experience.

5 Evaluation

Evaluation of the project based on the requirements defined at the beginning, as well as a discussion of the strengths and limitations of the solution.

5.1 Objective Achievement

Assessment of how well the project objectives were met, including the successful implementation of key features such as AR generation, AI querying, and document processing.

5.2 Requirement Satisfaction

Evaluation of the functional and non-functional requirements defined at the beginning of the project, including an analysis of how well the implemented solution satisfies these requirements.

5.2.1 Functional Requirements Evaluation

5.2.2 Non-Functional Requirements Evaluation

5.3 Strengths and Limitations

Discussion of the strengths of the solution, such as the integration of AR and AI technologies, the modular architecture, and the user-friendly interface. This section also addresses the limitations of the solution, including challenges with SAM segmentation, differences between true AR and native ARKit/ARCore implementations, and any other constraints encountered during development.

6 Further Improvements and Future Work

Suggestions for further improvements to the solution, as well as potential directions for future work. This may include enhancements to the AR generation process, improvements to AI response quality, additional features for the mobile and web applications, and exploration of new technologies or methodologies.

7 Professional, Ethical, Legal and Social Issues

Discussion of the professional, ethical, legal, and social issues related to the project, including considerations for data privacy, user consent, responsible AI usage, and the potential impact of the technology on society.